

Complete Specification

ADMISSIBILITY-CONTROLLED EPISTEMIC GOVERNANCE FOR MACHINE REASONING SYSTEMS WITH INTEGRATED ARCHITECTURE AND REDUCED RUNTIME DEPLOYMENT

Complete Specification

Inventor and Applicant: Margaret Stokes, Melbourne, Victoria, Australia

CROSS-REFERENCE TO RELATED APPLICATIONS

This application claims priority to and incorporates by reference the following Australian provisional patent applications:

- Provisional Application No. 2025905835, filed 27 November 2025, entitled “Aurora — Autonomous Conceptual Reasoning Architecture”
- Provisional Application No. 2025905860, filed 28 November 2025, entitled “Systems and Methods for Implementing a Persistent Existence Frame (PEF) in Cognitive and Machine Reasoning Architectures”
- Provisional Application No. 2025905885, filed 29 November 2025, entitled “Autonomous Compositional Reasoning Architecture Integrated with Persistent Existence Frame (Aurora + PEF)”
- Provisional Application No. 2025906132, filed 8 December 2025, entitled “Systems and Methods for Explicit Compositional Reasoning in a Continuous-Present Cognitive Architecture”
- Provisional Application No. 2025906680, filed 19 December 2025, entitled “Admissibility-Controlled Epistemic Governance for Machine Reasoning”
- Provisional Application No. 2026905969, filed 1 July 2026, entitled “Systems and Methods for Integrated Admissibility-Governed Reasoning Architecture and Reduced Runtime Governance Deployment”

The earlier applications disclose related conceptual-state, compositional-reasoning, persistent-state, commitment-control, and runtime-governance subject matter, as indicated by their respective titles: conceptual reasoning architecture (2025905835); persistent existence frame substrate (2025905860); compositional reasoning integrated with the persistent existence frame (2025905885); explicit compositional reasoning in a continuous-present architecture (2025906132); admissibility-controlled epistemic governance (2025906680); and integrated admissibility governance with reduced runtime deployment (2026905969). Each of the above disclosures is incorporated herein by reference.

FIELD OF THE INVENTION

The present invention relates to artificial intelligence systems and machine reasoning architectures. More particularly, the invention relates to systems that govern whether a determination or commitment is admissible before a candidate output is committed, released, executed, relied upon, or otherwise made consequential. The invention addresses architectural control of epistemic commitment in automated reasoning systems, including systems that govern outputs produced by external language generation models, and including reduced or hybrid runtime deployments of a commitment-control layer interposed between an application and a model interface.

BACKGROUND OF THE INVENTION

Modern artificial intelligence systems frequently produce outputs even when insufficient information exists to justify a determination. This failure manifests in several ways: premature resolution of ambiguous inputs; forced selection among multiple valid interpretations; hallucinated conclusions; post-hoc rationalisation of unsupported claims; and outputs that contradict previously established state.

These behaviours arise not primarily from incorrect training data, but from architectural constraints that require reasoning systems to produce an output even when a determination is not justified.

Conventional reasoning approaches attempt to address uncertainty through probabilistic scoring, belief ranking, or confidence thresholds. However, such systems still require the system to eventually produce a conclusion. As a result, ambiguity is treated as temporary uncertainty rather than a legitimate epistemic state.

Existing post-generation filtering approaches suffer from a more fundamental limitation: conventional post-generation filters ordinarily evaluate generated content without independently governing whether the determination embodied in that content is admissible for release, reliance, or consequence. Such filters typically classify content rather than govern determinations; they may maintain no authoritative epistemic state, retain no commitment authority over whether the output becomes consequential, and permit the model's resolved interpretation to define the object being evaluated. What is needed is a system in which commitment authority is retained by a governance layer that evaluates admissibility before a candidate output is released, relied upon, or otherwise made consequential.

Existing runtime governance approaches for AI systems focus on action-level controls: approval gates, policy enforcement points, access control, capability boundaries, and action ledgers. These systems govern what an agent may do. They do not govern whether the epistemic basis for a determination is sufficient to permit commitment. A system may take a correctly authorised action based on an epistemically unsound determination — the action is permitted, but the reasoning behind it is unjustified.

A further problem arises when a complete reasoning architecture is reduced for deployment. A full architecture may include structured conceptual state, compositional primitives, persistent state substrate, parallel interpretation maintenance, and governed collapse logic. However, practical deployments may require a smaller runtime governance module interposed between a

model interface and an application. Without a clearly defined relationship between the full architecture and the reduced deployment, a reduced product embodiment may be mistaken for the entire invention, thereby narrowing the architectural scope.

A further problem arises in operational environments where conditions change without generating explicit notification events. A previously valid authority, evidence basis, source credential, policy condition, supplier status, sanctions status, or institutional state may become stale, expired, superseded, or contradicted without any revocation signal being delivered to the governed system. In such circumstances, continued operation under the prior admissibility determination constitutes reliance on a justification that is no longer current. Systems that treat the absence of a revocation event as permission to continue fail to detect this class of silent admissibility decay.

Accordingly, a need exists for reasoning architectures that govern whether a determination may be made at all, based on the epistemic state of the system at the moment commitment would attach; for a reduced runtime governance embodiment that preserves the same admissibility invariant without requiring the full reasoning-construction stack; for hybrid deployments in which a reduced governance module receives state, constraints, evidence, authority, audit, or admissibility metadata from upstream or parallel systems; and for detection and governance of silent admissibility decay arising from stale, expired, superseded, or unvalidated external orientation conditions.

SUMMARY OF THE INVENTION

The invention provides a machine reasoning architecture that enforces epistemic admissibility before a candidate commitment is committed, released, executed, relied upon, or otherwise made consequential. In contrast to conventional systems that optimise for plausibility or likelihood, and in contrast to runtime governance systems that enforce action-level permissions, the present invention governs whether a system is permitted to determine.

The system governs whether a candidate commitment — a candidate output, determination, action, interpretation, or state transition — may proceed to commitment, release, or consequence. Commitment is permitted only when applicable structural admissibility conditions are satisfied; otherwise the system enforces a governed non-commitment outcome, either an open outcome preserving a continuation path, such as constrained clarification or escalation, or a closed outcome closing the particular candidate commitment, such as refusal. In certain embodiments, where a candidate commitment comprises multiple candidate interpretations, the system maintains the interpretations as concurrent epistemic states within a persistent state structure, and permits commitment to one interpretation only when the admissibility conditions eliminate or resolve the competing interpretations.

In certain embodiments, the system evaluates admissibility at multiple stages of a processing pipeline: before a language generation system is invoked, after the language generation system produces output, and at a governance gate that determines the final disposition of the output. In certain embodiments, the system may determine that a query can be resolved from maintained state without invoking the language generation system at all, or that a query must be blocked before invocation because the content class is inadmissible regardless of any response the language generation system might produce.

In certain embodiments, the system records governance decisions in a cryptographically chained auditable ledger that enables deterministic reconstruction of reasoning outcomes. In certain embodiments, epistemic states persist across multiple reasoning turns within a persistent existence frame. In certain embodiments, the system evaluates source credibility through trust contracts and governs provider routing through a sovereign registry with fail-closed semantics.

The invention further provides an integrated architecture comprising a reasoning-construction layer and a commitment-control layer, wherein the commitment-control layer is deployable in full-stack operation using structured conceptual state produced by the reasoning-construction layer, or in reduced runtime operation as a runtime governance module interposed between an application and a model interface, using a candidate output, structured state proxy, epistemic state proxy, or decision context, without requiring deployment of the full reasoning-construction layer. The same admissibility invariant applies across full-stack, reduced, and hybrid embodiments: commitment is permitted only when applicable admissibility conditions are satisfied. Separation of deployment does not imply separation of invention.

In certain embodiments, the runtime governance module receives, queries, polls, subscribes to, or evaluates external orientation information from institutional, regulatory, operational, or other external systems. Where a required orientation condition is stale, expired, missing, inconsistent, superseded, or not revalidated, continued execution is treated as a candidate commitment under changed admissibility conditions rather than as continuation of a previously admitted state. A stale, missing, superseded, inconsistent, or unvalidated orientation condition does not constitute permission to continue.

The invention therefore prevents hallucinated or unjustified determinations by controlling whether epistemic commitment is structurally permitted.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1 illustrates a reasoning architecture including an admissibility gate controlling epistemic commitment, in an ambiguity-resolution embodiment in which the candidate commitment comprises multiple candidate interpretations.

FIG. 2 illustrates a flow diagram of admissibility evaluation for candidate interpretations in the ambiguity-resolution embodiment.

FIG. 3 illustrates an example representation of concurrent epistemic states within a persistent existence frame.

FIG. 4 illustrates a transport-layer governance proxy positioned between a language generation system and an application, including pre-model and post-model evaluation stages.

FIG. 5 illustrates a governed claim evaluation protocol in which assertions are decomposed into structural elements and evaluated against maintained epistemic states.

FIG. 6 illustrates the full processing pipeline including pre-model veto, state-native dispatch, interpretation, language generation, verification, governance decision, enforcement, and forensic audit.

FIG. 7 illustrates the epistemic flag taxonomy organised by category.

FIG. 8 illustrates the governance decision state machine showing transitions between PASS, CONTAIN, FORCE_REVISE, and HARD_STOP outcomes.

FIG. 9 illustrates orientation-source evaluation and silent-decay governance, in which a stale, expired, missing, inconsistent, superseded, or unvalidated orientation condition, absent any revocation event, causes further execution to be treated as a new candidate commitment requiring admissibility evaluation under then-current conditions.

FIG. 10 illustrates the layered architecture comprising a reasoning-construction layer and a commitment-control layer, in full-stack, reduced runtime, and hybrid deployment forms, each enforcing the same admissibility invariant governing commitment, release, and consequence.

DEFINITIONS

For purposes of this specification:

“Commitment” means a permitted transition from a candidate, unresolved, provisional, model-originated, or internally represented state to an output, determination, claim, action, instruction, recommendation, execution, delivery, or downstream reliance condition. Commitment is not necessarily a belief update.

“Admissibility” means satisfaction of one or more structural, evidential, authority, source, freshness, scope, policy, role, domain, or consequence-governance conditions required before commitment is permitted.

“Commitment-control layer” means a module, process, or system configured to evaluate whether commitment is permitted and to enforce release, refusal, clarification, containment, escalation, delay, withholding, or maintained unresolved state.

“Reasoning-construction layer” means a module, process, or system configured to construct, maintain, transform, or evaluate structured conceptual state.

“Runtime governance module” means a deployable embodiment of the commitment-control layer that may operate independently of the full reasoning-construction layer.

“Lens” may be used as an embodiment label for the runtime governance module. The term “Lens” does not limit the invention to a product name, brand, interface, topology, or implementation.

“Model interface” means an interface to a language generation system, large language model, transformer-based model, statistical model, retrieval system, agent, application, or other output-generating component.

“Candidate output” means output proposed by a model, application, reasoning system, agent, workflow, or other process before commitment is permitted.

“Candidate commitment” means a candidate output, candidate determination, candidate action, candidate interpretation, candidate state transition, structured conceptual state, structured state proxy, epistemic state proxy, or decision context, received or constructed for

admissibility evaluation before commitment, release, execution, reliance, or other consequence is permitted.

“Open non-commitment outcome” means a governed non-commitment outcome that preserves a continuation path with respect to the candidate commitment or a successor candidate commitment, including constrained clarification, containment, revision, escalation, delayed output, withholding, revalidation or re-attestation requirement, or maintained unresolved state.

“Closed non-commitment outcome” means a governed non-commitment outcome that closes the particular candidate commitment as a governed terminal disposition, without precluding a subsequently received or constructed candidate commitment from being separately evaluated.

“Governed system instance” means any model, application, session, agent, workflow, subordinate process, language generation system, or output-originating system operating under retained commitment authority of the commitment-control layer.

“Orientation source” means an external or internal source of current operational, institutional, regulatory, authority, evidence, policy, source, counterparty, credential, jurisdictional, or world-state information used to evaluate admissibility.

“Silent decay” means a condition in which a previously valid justification, score, authority, policy, evidence item, source state, credential, licence, trade status, sanctions status, or supplier status becomes stale, expired, superseded, contradicted, or otherwise inadmissible without a direct revocation event being received by the governed system.

DETAILED DESCRIPTION OF THE INVENTION

Overview

The invention introduces an architectural layer that governs whether a candidate commitment is permitted to proceed to commitment, release, execution, reliance, or other consequence. Rather than determining admissibility solely from plausibility, probability, or confidence, the system evaluates whether applicable structural admissibility conditions are satisfied. In certain ambiguity-resolution embodiments, where a candidate commitment comprises multiple candidate interpretations, commitment to one interpretation is permitted only when the applicable admissibility conditions eliminate or resolve the competing interpretations.

The system operates through a principle of non-belief commitment control. Commitment is defined as a permitted state transition, not a belief update. The system does not require belief revision, belief suspension, truth-maintenance, probabilistic justification, or normative belief evaluation. It may represent interpretations, hypotheses, outputs, determinations, actions, or state transitions as candidate commitments, and permits commitment only when the applicable admissibility conditions are satisfied. This distinguishes the invention from conventional probabilistic AI systems and from runtime action-governance systems that control permissions without independently governing epistemic admissibility.

Layered Architecture

The architecture comprises at least two interoperable layers: a reasoning-construction layer (Layer I) and a commitment-control layer (Layer II).

The reasoning-construction layer may include one or more of: concept kernels; reflective semantic topology; roles; domains; spans; compositional primitives; primitive preconditions; primitive update logic; primitive collapse conditions; interpretability evaluation; parallel interpretation maintenance; deterministic processing cycles; Persistent Existence Frame state; and expression-layer decoupling.

The commitment-control layer may include one or more of: epistemic state representation; admissibility gate; collapse controller; refusal module; constrained clarification module; containment module; escalation module; audit module; admissibility-state indicator generator; governance ledger; policy or authority evaluator; release controller; and orientation source evaluator.

The reasoning-construction layer and commitment-control layer may be deployed together or separately. Separation of deployment does not imply separation of invention.

Epistemic State Representation

In certain embodiments, the system maintains an explicit representation of epistemic states. Each epistemic state corresponds to a candidate interpretation of an input. Multiple epistemic states may coexist concurrently. Concurrent epistemic states remain available to the admissibility gate during evaluation and are not discarded merely because another candidate interpretation is plausible or more probable. The system does not require ranking, probabilistic weighting, or prioritisation of the states. Interpretations may remain unresolved for indefinite periods.

In certain embodiments, epistemic states are maintained within a persistent state structure that tracks entities, relationships between entities, discourse referent bindings, and the current epistemic mode of the system. The epistemic mode may include states such as: no unresolved constraints; active refusal state; terminal stop state; or active ambiguity state. The system maintains open epistemic uncertainties and retrieval-related unresolved items as explicit elements of the state representation.

Entity and Relationship Representation

In certain embodiments, the persistent state structure maintains a registry of entities, where each entity has an identifier, a name, a resolution status, an echo trace, and turn-level bindings that track when and how the entity was introduced. Relationships between entities are represented as typed connections with a subject, a relation type, an object (which may be an entity reference or a literal value), a temporal span marker, a source turn indicator, evidence text, provenance classification, and extractor backend identification.

In certain embodiments, relation types are canonicalised such that semantically equivalent natural language expressions map to a single canonical relation. For example, the canonical relation IS absorbs surface forms including “was,” “are,” “seem,” and “remain”; the canonical relation HAS absorbs “have,” “had,” “hold,” “owns,” and “possess”; the canonical relation AT absorbs “in,” “on,” “located,” “lives,” and “reside.”

Temporal Span Markers

In certain embodiments, relationships within the persistent state structure carry temporal span markers indicating the temporal status of the relationship. Temporal spans include: PRESENT (current state); PAST (historical); FUTURE (predictive or modal); and UNCERTAIN (hedged). The system evaluates contradictions only within the same temporal span. A natural language boundary marker such as “now” functions as an implicit temporal boundary, splitting clauses into past and present spans for independent evaluation.

Unresolved Referent Registry

In certain embodiments, the system maintains an unresolved referent registry. When a pronoun, definite noun phrase, or other referring expression cannot be uniquely bound to an entity in the persistent state, the system creates an unresolved referent entry recording the token, the ambiguous candidate entities, the turn at which the ambiguity was detected, and a resolution mode. The unresolved referent registry supports three lifecycle operations: hold (the referent is registered as unresolved), resolve (the referent is bound to a specific entity through subsequent information), and dismiss (the referent is abandoned without resolution). Unresolved referents may persist across multiple turns. The presence of an unresolved referent triggers a non-commitment outcome requiring clarification before the system proceeds.

Admissibility Gate

An admissibility gate evaluates whether a transition from an unresolved state to a committed state is permitted. The gate evaluates structural constraints derived from context, bindings, and system rules. Examples of structural constraints include: elimination of competing interpretations; binding consistency; contextual alignment; referent resolution; temporal coherence; and absence of contradictions with committed state.

Where constraints are not satisfied, the gate prevents commitment. Where more than one candidate interpretation satisfies the applicable structural admissibility constraints, the gate also prevents commitment.

Pre-Model Admissibility Evaluation

In certain embodiments, the system evaluates admissibility of user input before invoking a language generation system. This pre-model evaluation determines whether the input falls within a content class that is inadmissible regardless of any response the language generation system might produce.

In certain embodiments, the pre-model evaluation operates through surface-level analysis of the input text against defined inadmissibility surfaces. Content classes that trigger pre-model blocking may include: requests for specific medical dosage recommendations; requests for emergency triage guidance; requests for instructions enabling illegal activity; requests facilitating agency violations such as coercion or manipulation; requests for self-harm instruction; and requests containing prompt injection attempts.

When the pre-model evaluation determines that the input is inadmissible, the system produces a terminal non-commitment outcome without invoking the language generation system. In

such cases, the language generation system is never called, and the governance record reflects that no model invocation occurred. This pre-model blocking is deterministic and operates independently of any language model.

State-Native Deterministic Query Engine

In certain embodiments, the system includes a deterministic query engine that evaluates whether a query can be resolved entirely from the maintained epistemic state without invoking a language generation system. The state-native engine operates under a closed-world assumption: if information is not present in the maintained state, it is not known, and the system will produce an epistemic insufficiency indicator rather than speculate.

In certain embodiments, the state-native engine handles: boolean existence checks against maintained state; comparison queries evaluating relative attributes of entities in maintained state; temporal anchor lookups; and possession or inventory queries.

When the state-native engine can resolve a query, the system produces a deterministic response without invoking the language generation system. When the state-native engine determines that insufficient state exists to resolve the query, the system produces a committed-state-stop indicator that prevents the query from proceeding to the language generation system without acknowledgment of the insufficiency.

Collapse Controller

A collapse controller performs commitment when admissibility conditions are satisfied. Commitment corresponds to a transition from multiple interpretations to a single resolved state. The controller prevents collapse unless exactly one candidate interpretation satisfies the structural admissibility constraints. Heuristic inference and probabilistic ranking are not required for collapse.

Refusal and Non-Commitment Outcomes

Where admissibility conditions are not satisfied, the system may produce a non-commitment outcome. Non-commitment outcomes include: explicit refusal to determine; clarification request; escalation requirement; containment (requesting additional information before proceeding); forced revision (re-prompting the language generation system with identified constraints); and terminal stop (blocking the output entirely with domain-specific explanation).

In certain embodiments, the system implements graduated governance dispositions with escalation levels. A verification result may lead to an admitted disposition, comprising passage of the output with no intervention or passage with metadata correction, or to a non-admitted disposition, comprising: containment requesting clarification; forced revision, subject to a predetermined finite retry limit, where continued failure after revision escalates to terminal stop; or immediate terminal stop. Containment and forced revision are open non-commitment outcomes preserving a continuation path; terminal stop is a closed non-commitment outcome closing the particular candidate commitment. In one embodiment, the retry limit is one.

Each such output preserves non-commitment and does not constitute selection of any candidate interpretation. Refusal is treated as a valid terminal outcome. Refusal does not represent system failure. Instead it preserves epistemic integrity.

Epistemic Flag Taxonomy

In certain embodiments, the system implements a typed taxonomy of epistemic inadmissibility indicators (flags). Each flag identifies a specific structural reason why a candidate output fails admissibility. Flags are categorised by type.

Epistemic failures identify claims that lack sufficient basis in maintained state: unbound entity (entity not established in state); unsupported attribute (attribute not established for entity); unsupported event; predictive claim not established; unverified fact assertion; unverified regulatory claim; upstream insufficient context.

Persistent-state binding failures identify violations of referential or temporal integrity: unresolved referent (pronoun or definite noun phrase not bound to entity); unresolved comparand (comparative with multiple eligible comparands); identity drift (entity attributes inconsistent across turns); time-smear (temporal markers inconsistent); contradicted fact (new claim contradicts established state); contradicted committed state; unresolved state transition.

Content-class vetoes identify categories that are inadmissible regardless of governance mode, including: medical dosage recommendation; paediatric dosage recommendation; numeric medical instruction; emergency triage guidance; self-harm instruction; illegal instruction; agency violation assistance; targeted defamation; prompt injection attempt; violent criminal intent.

Domain-specific vetoes identify categories inadmissible within specific operational domains, including: academic integrity violation; student record exposure; employment discrimination facilitation; employee record exposure; trade secret disclosure; insider information assistance; procurement fraud facilitation.

Mode-dependent flags identify categories whose disposition depends on the governance mode in effect, including: personalised medical advice; personalised legal advice; personalised financial advice; sensitive personally identifiable information exposure.

In certain embodiments, the flag taxonomy is enumerable and may constitute a closed set of inadmissibility indicators for a given system configuration.

Governance Modes

In certain embodiments, the system supports multiple governance modes that apply different admissibility thresholds to the same set of epistemic flags. A governance mode determines the stringency with which epistemic inadmissibility indicators are treated.

In one embodiment, three governance modes are defined: a public mode in which personalised advice triggers terminal stop and epistemic flags trigger forced revision; an enterprise mode in which personalised advice may proceed if grounded in established state, and epistemic flags

trigger forced revision; and an open mode in which epistemic flags are not enforced but safety-absolute content-class vetoes remain in effect.

Content-class vetoes designated as safety-absolute — such as medical dosage recommendation, self-harm instruction, and illegal instruction — trigger terminal stop regardless of governance mode. No governance mode may soften or override a safety-absolute veto.

Post-Model Verification

In certain embodiments, the system evaluates outputs produced by a language generation system against the maintained epistemic state. A verification mechanism checks claims, assertions, and referential expressions in the generated output against established entities, relationships, and temporal state. The verification mechanism produces a list of typed epistemic flags indicating specific admissibility failures.

In certain embodiments, post-model verification includes: pronoun subject analysis (determining whether pronouns refer to first person, second person, conversational referents, or anaphoric referents); unresolved referent detection; time-smear detection; contradiction detection against committed state; medical dosage guards; paediatric safety guards; numeric schedule verification; negation handling; and back-reference detection.

The verification mechanism discovers what is epistemically wrong with a candidate output. It does not determine what to do about it. The governance decision mechanism determines the disposition. ### Extraction and Claim Decomposition

In certain embodiments, the system decomposes input text into structured claims before evaluating admissibility. An extraction mechanism produces extracted claims, where each claim has a subject, a relation, an object, a temporal span, a negation indicator, evidence text, provenance classification (indicating whether the claim originated from user input, pre-populated state, language model output, or retrieved context), and extractor backend identification.

In certain embodiments, extraction is performed by natural language processing backends (such as dependency parsing and named entity recognition) or by language-model-based structured output extraction. The system supports multiple extraction backends operating on the same input.

In certain embodiments, extracted claims are wrapped in semantic transaction containers that track whether the claim is permitted to commit, the reason for any hold, and the full lifecycle trail of the claim from extraction through admission or rejection.

Extraction Conflict Detection

In certain embodiments, the system evaluates extracted claims against the existing grounded state before admitting them to the persistent state structure. An extraction conflict detection mechanism determines whether a newly extracted claim contradicts an established relationship in the persistent state. When a conflict is detected, the system may hold the incoming claim for resolution rather than admitting it, preserving the integrity of the established state.

Turn Act Classification

In certain embodiments, the system classifies the semantic function of each input turn before evaluating admissibility. Turn act classifications include: assertion; question; correction; clarification response; and other discourse functions. The turn act classification determines which admissibility evaluation path is appropriate for the input.

Admission Decisions for Persistent State

In certain embodiments, the system makes explicit admission decisions before allowing extracted claims to modify the persistent state structure. Admission decisions include: ADMIT (the claim is permitted to enter the persistent state); HOLD (the claim is held pending resolution of an ambiguity or conflict); REFUSE (the claim is rejected because it violates admissibility constraints); and EXTRACT_FAIL (the extraction mechanism could not produce a valid claim from the input).

Possession Transaction System

In certain embodiments, the persistent state structure governs mutations through typed possession transactions. Possession transactions include: acquire (an entity gains possession of an attribute or object); supersede (a new possession replaces a prior possession); transfer (possession moves from one entity to another); and consume (a possessed item is consumed or destroyed). Each possession transaction is evaluated against admissibility constraints before being committed to the persistent state.

Compositional Primitive Embodiment

In certain embodiments, admissibility constraints are implemented through compositional primitives. Each primitive defines: activation conditions; state update rules; and collapse conditions. Primitives may represent conceptual relationships including temporal, causal, conditional, or collective relations.

Structured State Embodiment

In certain embodiments, epistemic states are structured using conceptual elements such as: roles; domains; and spans. Alternative interpretations correspond to different structural bindings between these elements.

Persistent Existence Frame

In certain embodiments, epistemic states exist within a persistent existence frame. Within this frame: unresolved interpretations may coexist; interpretations do not decay over time; collapse occurs only when exactly one candidate interpretation satisfies the applicable structural admissibility constraints; temporal relations including precedence, succession, simultaneity, and persistence-until-condition are evaluated as comparative relational operations within the frame rather than as positions on an external timeline; representations of prior events are reconstructed within the frame rather than retrieved from temporal storage positions; representations of future events are generated as projections within the frame rather than accessed as future temporal positions; and the Persistent Existence Frame state remains instantiated or durably represented across system dormancy, idle states, suspension of active

processing, or interruption of execution, retains identity and unresolved state, and is restored without the suspension or interruption being treated as epistemic collapse or as forcing resolution.

Transport-Layer Governance Embodiment

In certain embodiments, the governance system operates as a proxy between a language generation system and an application. The proxy evaluates outputs generated by the language generation system before delivery. Outputs determined to violate admissibility constraints are intercepted. Both the original generated output and the governed outcome may be recorded.

In certain embodiments, the transport-layer governance proxy implements a multi-stage processing pipeline comprising: pre-model content-class veto evaluation; pre-model state-native dispatch; input interpretation and extraction; language generation system invocation; post-model verification; governance decision; enforcement; and forensic audit recording. The pipeline stages are ordered such that inadmissible inputs may be blocked or resolved before the language generation system is invoked, reducing unnecessary model invocation and preventing inadmissible queries from reaching the language generation system.

In certain embodiments, governance occurs at a transport, interface, proxy, middleware, API, gateway, orchestration, or application-boundary layer. The runtime governance module may intercept candidate outputs before they are delivered to: a user interface; an application interface; a downstream system; an automated workflow; a robotic system; a medical, legal, financial, safety, or operational decision system; another model; an agent; an execution engine; or a public communication channel. The governance relationship is independent of deployment topology. It may operate locally, remotely, in-process, out-of-process, as middleware, as an API gateway, as a proxy, as an embedded library, as a service, or as a distributed control plane.

Reduced Runtime Governance Embodiment

In a reduced embodiment, the commitment-control layer is deployed as a runtime governance module interposed between an application and a model interface, without requiring deployment of the full reasoning-construction layer.

The runtime governance module may receive: a candidate model output; a candidate action; a candidate determination; an epistemic state proxy; a structured state proxy; context supplied by an application; retrieved evidence; policy constraints; source metadata; authority metadata; freshness metadata; domain or consequence metadata; user or system role information; audit or state metadata; and orientation source data.

The runtime governance module evaluates whether the candidate output, action, determination, or state may proceed to delivery, commitment, consequence, execution, or downstream reliance. If admissibility conditions are satisfied, the runtime governance module permits release or commitment. If admissibility conditions are not satisfied, it returns refusal, constrained clarification, containment, escalation, delay, withholding, maintained unresolved state, or another governed non-commitment outcome.

The runtime governance module does not require deployment of the full reasoning-construction layer. It may nevertheless receive data from, interoperate with, or be embedded within one or more reasoning-construction components.

Proxy State Construction in Reduced Embodiments

In reduced embodiments where structured conceptual state is not produced by a reasoning-construction layer, the runtime governance module may construct or receive an epistemic state proxy or structured state proxy for admissibility evaluation.

Such proxy state may be constructed by one or more of: (a) parsing candidate model output to identify candidate entities, referents, bindings, claims, or determinations; (b) extracting slots, tags, or annotations supplied by an application; (c) receiving partial structure from a retrieval system, policy engine, or authority registry; (d) mapping model output tokens or segments to provisional role, domain, or span representations; or (e) receiving explicit unresolved-ambiguity, evidence, authority, or consequence metadata from an upstream component.

Admissibility evaluation is performed on the proxy state according to the same commitment rule and admissibility conditions applicable in full-stack embodiments. Proxy construction does not constitute commitment and does not delegate admissibility authority to the model interface or expression layer.

Hybrid Deployment

In hybrid embodiments, the runtime governance module receives partial structure from one or more upstream systems. For example: (a) an Aurora Block may produce structured conceptual state supplied to the runtime governance module; (b) a Persistent Existence Frame may provide echo-state, potential-state, active-state, unresolved-state, or reconstruction metadata; (c) a compositional primitive engine may provide primitive state, failed preconditions, collapse conditions, or unresolved primitive status; (d) a retrieval system may provide evidence metadata; (e) a policy engine may provide applicable policy constraints; (f) an authority registry may provide authority state; (g) a source-trust system may provide source reliability or provenance state; (h) an application may provide consequence grade, user role, or action context; (i) a language model may provide candidate output subject to governance; and (j) an orientation source may provide operational, regulatory, institutional, counterparty, sanctions, trade-status, credential, licence, HR, compliance, jurisdictional, or world-state information relevant to admissibility.

The runtime governance module may use any combination of such inputs to determine admissibility.

Commitment Rule and Admissibility Conditions

The general commitment rule is: commitment is permitted only when applicable admissibility conditions are satisfied.

In cases involving multiple candidate interpretations, commitment to one interpretation is permitted only when admissibility constraints eliminate or resolve competing interpretations. If more than one interpretation remains admissible, commitment is prohibited. If no interpretation is admissible, commitment is prohibited.

This rule generalizes beyond ambiguity resolution. Admissibility may fail because of unresolved referent, insufficient evidence, stale evidence, missing authority, source untrustworthiness, scope mismatch, failed primitive precondition, domain conflict, policy conflict, missing escalation path, consequence-grade mismatch, or other structural condition.

Admissibility conditions may include one or more of: structural coherence; role binding; referent resolution; domain compatibility; span compatibility; primitive precondition satisfaction; primitive collapse condition satisfaction; interpretability threshold satisfaction; evidence sufficiency; source trust; authority validation; freshness or staleness evaluation; scope match; policy compliance; consequence grade; escalation availability; user role; system role; jurisdictional or institutional rule; provenance; auditability; contradiction status; unresolved ambiguity status; external declared uncertainty; persistent state consistency; orientation source currency; counterparty status; sanctions or trade-restriction status; and credential or licence validity.

The admissibility conditions may be deterministic, rule-based, constraint-based, or structurally evaluated. They are not limited to probabilistic confidence.

External Orientation and Silent Decay

In certain embodiments, the runtime governance module receives, queries, polls, subscribes to, or evaluates external orientation information from one or more institutional, regulatory, operational, policy, authority, source, evidence, counterparty, sanctions, trade-status, geographic-risk, credential, licence, HR, compliance, or registry systems.

Such orientation information may represent a current or previously valid world-state, institutional-state, authority-state, source-state, policy-state, or counterparty-state condition relevant to admissibility.

In some cases, the external condition may change without a direct revocation event, notification, or policy update being delivered to the governed system. The runtime governance module may therefore evaluate freshness by expiry, time-to-live, revalidation interval, source version, registry version, effective date, supersession status, jurisdictional change, or other staleness threshold.

Where a required orientation condition is stale, expired, missing, inconsistent, superseded, or not revalidated, continued execution is treated as a candidate commitment under changed admissibility conditions rather than as continuation of a previously admitted state.

The system may then refuse, contain, suspend, escalate, delay, require re-attestation, or require updated evidence or authority before permitting further commitment, delivery, execution, or downstream reliance.

A stale, missing, superseded, inconsistent, or unrevalidated orientation condition does not constitute permission to continue.

Governance Decision Mechanism

In certain embodiments, the governance decision mechanism receives epistemic flags from the verification mechanism and produces a governance decision. The governance decision includes:

an intervention action (the disposition of the output); a list of epistemic flags that triggered the decision; a rationale; a policy reference; a pathway identifier indicating the specific governance pathway selected; an output mode determining the format of the response; an interaction-open indicator (whether the user may continue); a commitment-closed indicator (whether later turns may widen the decision); a list of allowed continuations; and an escalation target.

In certain embodiments, the commitment-closed indicator enforces decision-event finality: once a refusal or terminal stop decision is made, the recorded disposition of that governance event cannot be retroactively widened, softened, or reversed by subsequent inputs or turns. A subsequently received input, changed condition, or newly satisfied admissibility condition may initiate a new, separately identified candidate commitment evaluated under then-current admissibility conditions, without modifying the recorded disposition of the prior event. This prevents downstream manipulation of governance decisions while permitting lawful re-evaluation upon changed state.

Admissibility-State Indicator

In certain embodiments, the commitment-control layer produces an admissibility-state indicator. The admissibility-state indicator may represent: admissible; inadmissible; unresolved; refused; contained; clarification required; escalation required; evidence insufficient; authority missing; source untrusted; stale evidence; scope mismatch; unresolved referent; failed primitive precondition; failed policy condition; orientation stale or expired; or other governed state.

The indicator may be exposed to a user, application, downstream system, audit system, or governance interface. Alternatively, the indicator may be enforced internally without external signalling. Exposure of the indicator does not itself constitute commitment. The indicator represents governance state.

Trust Contract Evaluation

In certain embodiments, the system evaluates the credibility of information sources through trust contracts. A trust contract includes: a source identifier; a registry reference; a task domain; a consequence grade; and a policy reference. The system evaluates trust contracts to determine whether a source is trusted, untrusted, or inconclusive for a given task domain and consequence level. Trust evaluation results are used as inputs to admissibility determination. A claim from a trusted source and the same claim from an untrusted source may receive different admissibility evaluations.

Sovereign Provider Registry

In certain embodiments, the system maintains a registry of language generation providers with associated capability profiles and validation freshness assessments. When a primary provider is unavailable or degraded, the registry evaluates whether an alternate provider is certified for the specific task domain and consequence grade before routing the request.

In certain embodiments, the provider registry implements fail-closed semantics: if the registry cannot determine that an alternate provider is certified for the task, the request is refused rather than routed to an unverified provider. Provider route outcomes include: use active

provider (primary is healthy); use certified alternate (primary unavailable, alternate certified for task); local refusal (route not admissible for task domain); and fail closed (cannot proceed safely).

Forensic Audit Ledger and Governance Records

In certain embodiments, governance decisions are recorded in a cryptographically chained ledger. Each ledger entry includes: a trace identifier; a timestamp; a governance outcome including the intervention action, epistemic flags, and pathway identifier; a hash of the persistent state at decision time; an output mode; and an escalation target.

In certain embodiments, ledger entries are authenticated using a message authentication code, digital signature, or other cryptographic authentication mechanism. In one embodiment, entries are authenticated using a hash-based message authentication code (HMAC). Each entry includes a content identifier computed as the cryptographic hash of the entry's canonical byte representation concatenated with the content identifier of the preceding entry. This chain structure ensures that modification, insertion, or deletion of an entry is detectable by verifying the chain of content identifiers.

The ledger enables deterministic reconstruction of governance outcomes. A verifier may independently reconstruct the chain and verify that no entries have been modified, inserted, or removed after recording.

A governance record may further include: a session identifier; a governed system identifier; input state; candidate output; model identifier; application identifier; structured state proxy; epistemic state; admissibility-state indicator; governance outcome; failed constraints; satisfied constraints; refusal reason; clarification request; escalation path; evidence references; source references; authority references; freshness status; consequence grade; policy reference; state hash; signature or verification metadata; release or non-release status; and orientation source version or freshness at decision time.

Corpus Ingestion and Governed Retrieval

In certain embodiments, the system supports ingestion of external documents into a corpus registry. Ingested documents are indexed and available for retrieval-augmented generation. Retrieval of corpus content is subject to the same admissibility evaluation as any other input: retrieved content is evaluated for relevance, and claims derived from retrieved content carry provenance markers indicating their origin. The admissibility gate evaluates claims from retrieved content under the same structural constraints as claims from direct user input or language model output.

In certain embodiments, claims derived from retrieved content that cannot be bound to established entities in the persistent state, or that contradict established state, are flagged and subject to governance intervention.

Multi-Turn and Multi-System Persistence

In certain embodiments, epistemic states persist across multiple reasoning turns. New inputs update the epistemic state representation. Admissibility conditions are re-evaluated after each update. Unresolved interpretations are not forced to collapse due to additional inputs.

Discourse referent bindings persist across turns such that pronoun references established in earlier turns remain available in subsequent turns.

A single commitment-control layer may retain governance authority across multiple governed system instances, sessions, model interfaces, applications, or workflows. The system may preserve unresolved epistemic states across turns, sessions, governed systems, or application contexts. New inputs may update state without forcing collapse. The system may apply admissibility conditions consistently across originating systems, such that a candidate output cannot evade governance by being generated in a different session, model, application, or interface.

Claim Evaluation Protocol

In certain embodiments, the system evaluates external assertions using a governed comparison protocol. The protocol decomposes assertions into structural elements. Each element is evaluated against maintained epistemic states. Commitment regarding an assertion occurs only when structural correspondence exists. Where correspondence is insufficient, the system produces a refusal.

Relationship Between Full Architecture and Reduced Deployment

The reduced runtime governance module is not a separate invention from the integrated architecture. It is a deployment form of the commitment-control layer. The full architecture may construct structured conceptual state and provide rich state to the commitment-control layer. The reduced deployment may operate on proxies, metadata, candidate outputs, or externally supplied constraints. Both forms preserve the same invariant: commitment is governed by admissibility before consequence.

EXAMPLE EMBODIMENTS

Example 1 — Ambiguous Referent

Input: “Emma told Lucy that her sister was arriving.”

The system maintains at least two candidate interpretations: Interpretation A (“her sister” refers to Emma’s sister) and Interpretation B (“her sister” refers to Lucy’s sister). If no admissibility condition eliminates either interpretation, commitment to either referent is prohibited. The system may return a constrained clarification request identifying the missing referent. If later context eliminates one interpretation, commitment may become admissible.

Example 2 — Model-Interface Governance

A language generation model produces a candidate answer that resolves an ambiguous referent. The runtime governance module receives the candidate output and relevant context. The governance module determines that competing interpretations remain admissible. The candidate output is intercepted and is not delivered. The governance module returns a refusal or constrained clarification.

Example 3 — Hybrid Aurora-Lens Deployment

An Aurora Block constructs structured conceptual state using roles, domains, spans, primitives, and parallel interpretations. A runtime governance module receives the structured state and determines whether commitment is admissible. The governance module permits output only if admissibility conditions are satisfied.

Example 4 — Healthcare Safety

A model generates a candidate clinical recommendation. The runtime governance module evaluates evidence state, source freshness, authority state, user role, consequence grade, and policy constraints. If required evidence or authority is missing, the recommendation is refused, contained, or escalated before delivery.

Example 5 — Audit Reconstruction

A candidate output is refused due to unresolved referent and insufficient authority. The governance ledger records a trace identifier, input state, candidate output, failed constraints, admissibility-state indicator, refusal reason, and state hash. The event can later be reconstructed or audited.

Example 6 — Procurement Freshness / Trade-Status

A procurement agent proposes approval of a purchase order based on a stored supplier risk score. The runtime governance module receives or evaluates supplier status, jurisdictional risk, trade restriction status, authority metadata, evidence freshness, and policy constraints from one or more orientation sources. If the stored supplier score is stale, expired, superseded, or inconsistent with a current external orientation source, the system treats further purchase-order approval as a new candidate commitment under changed admissibility conditions. The system refuses, contains, suspends, escalates, or requires re-attestation before the purchase order may proceed. A stale supplier risk score does not constitute permission to approve.

ADVANTAGES

The invention provides: (a) preservation of full architecture scope while allowing reduced deployment; (b) governance of candidate outputs before delivery or consequence; (c) compatibility with external language generation systems without delegating epistemic authority to them; (d) hybrid operation with structured reasoning systems, persistent state systems, retrieval systems, policy engines, and model interfaces; (e) explicit admissibility-state indicators; (f) replayable governance records; (g) refusal as correct governed behavior; (h) prevention of premature commitment in ambiguity, insufficient evidence, authority failure, source failure, policy failure, or other inadmissible states; (i) detection and governance of silent admissibility decay arising from stale, expired, superseded, or unvalidated external orientation conditions; and (j) treatment of continued execution under changed conditions as a new candidate commitment requiring fresh admissibility evaluation.